Gömülü Sistem Alıştırmaları

Sürüm 0 pre-alpha

IBeRyUS (Ibrahim)

03 Haz 2020

İçindekiler

1 İçindekiler **3**
1.1 Giriş . . . 3
1.2 C Programlama Dili . . . 4
1.3 MSYS2 Kurulumu . . . 15
1.4 GNU Arm Toolchain Kurulumu . . . 19
1.5 Arduino Geliştirme Ortamı . . . 19
1.6 7 Segment BCD . . . 21
1.7 7 Segment Tarama . . . 29
1.8 Geliştirme Ortamı . . . 32
1.9 Blink (Göz kırpma) . . . 33

Dizin **35**

Gömülü sistemlerin C dili ile programlanmasını ve özellikle gömülü sistemlere özel kullanımları incelemeye ve elimden geldiğince örneklendirerek anlatmaya çalışacağım. İçerik dili Türkçe'dir fakat amacım programlama dillerinde kullanılan tüm terimlerin Türkçe karşılığını bulmak değildir. Bu yüzden bazı terimleri İngilizce olarak kullanmış olabilirim.

İçerikte olabilecek imla hataları ve düşük cümleler gibi aksaklıklardan ötürü şimdiden affınıza sığınıyorum. Bu tarz düzeltilmesi gereken noktaları github üzerinde pull-request, issue, direk mesaj veya e-posta atarak bildirirseniz gerekli düzeltmeler yapılacaktır.

Buradaki uygulama örneklerindeki kodlar örnek olarak yazılmış kodlardır ve ufak hatalar(bug) içerebilir. Direk kopyala yapıştır ile projenizde kullanmak yerine yöntemleri öğrenerek kendi kodlarınızı yazmanızı tavsiye ederim.

Özellikle yeni başlayanlara C diline ait kısa bir giriş ve gömülü sistemlerde uygulanan yöntemleri anlatmaya çalıştığım bölümden okumaya başlamanızı tavsiye ederim.

BÖLÜM 1

İçindekiler

1.1 Giriş

Not: Buradaki örneklerde ya da açıklamalarda C dili anlatılmayacak, daha çok C dilinin uygulamaları üzerine bilgiler verilecektir. Buradaki uygulamaları incelemek ve anlamak için temel elektronik, mikrodenetleyici donanım, algoritma ve C programlama dili bilgilerine sahip olmanız gerekmektedir. Verilen kodların satır satır açıklamaları yapılmayacak, şemalarda çok fazla detaya inilmeyecektir.

1.1.1 Gömülü Sistem Ne Demektir

Gömülü sistemler kısaca sistemin çalışması için gereken tüm donanım ve yazılım ekipmanların bir arada bulunduğu sistemlerdir. Gömülü sistemler genel kullanımdan ziyade ihtiyaç duyulan göreve/işe göre özel olarak tasarlanırlar. Gömülü sistemleri buradaki bir kaç satır ile anlatmak maalesef mümkün değildir. Daha detaylı bilgiler için diğer kaynakları araştırmanızı tavsiye ederim.

Gömülü sistem örneklerinde genel olarak bare-metal(herhangi bir işletim sistemi veya iş yönetim sistemi kullanmadan direk yazılımın mikrodenetleyici üzerinde çalışığı) örneklere yer vereceğim. Bare-metal uygulamalarda kullanılan programlama dilleri genellikle assembler ve C/C++ dilleridir. Buradaki örnekler C dilinde yazılmış olacaktır.

1.1.2 Ekstra Bilgiler

Özellikle birden fazla kişinin birlikte çalıştığı projelerde kodlama standardı uygulanması proje yönetimi ve kod anlaşılabilirliğini arttırır. Kodlama yapılırken sadece kodlama standardı değil farklı standartlara uygunluk da kontrol edilebilir. Örneğin güvenlik öncelikli(İng.Safety-critical) sistemlere ait kodlarda MISRA standartlarına uygunluk da kontrol edilir. Örnek kodlama standardı olarak Barr Kodlama Standardı'na bakabilirsiniz.

Gömülü sistemler örneklerinde kolaylıkla satın alınabilecek geliştirme kartları üzerinden örnekler vermeye çalışacağım. Özellikle Arduino ve STM32 kartları kolayca ve ucuza bulunabilmektedir. Gömülü

sistemlerde bu tarz kartları yeni donanımları öğrenmek ve hızlıca testler yapmak için kullanabilirsiniz. Genel olarak bu yaklaşımı uygulamıyorum. Önce şema ve ilk prototip kartını hazırlayıp ondan sonra sürücü fonksiyonları ile başlıyorum. Kullanacak olduğunuz mikrodenetleyiciye ait donanımları okumadan, özelliklere bakmadan ezbere kod yazmaya başlamak ileride size sorun yaratabilir. Bazen bir pin değişikliği ile yüzlerce satır koddan kurtulabilir, daha hızlı cevaplar üretebilirsiniz.

1.2 C Programlama Dili

Buradaki açıklamaları kolay anlamak için en az bir programlama dilini bilmeniz gereklidir. Eğer C diline ait bir bilginiz yok ise tavsiyem C programlama dilini anlatan bir kitabı okumanızdır. Ben C diline Rifat Çölkesen - C Programlama Dili'in yazmış olduğu C kitabını okuyarak başlamıştım.

Gömülü sistemlerde genel olarak ana iki programlama mimarisinden biri seçilir. Bunlar:

- Bare-metal (direk firmware hazırlamak)
- Gerçek zamanlı işletim sistemi(RTOS) üzerinde çalışmak

Genel olarak ben bare-metal programlama tercih ediyorum. Şimdilerde gömülü sistemlerde kullanılan mikrodenetleyiciler eski mikrodenetleyicilerden daha fazla RAM ve ROMa sahip olduğu ve mikroişlemciler gibi RAM stack, software interrupt, vb. ilave imkanlar sağladığı için RTOS kullanımı giderek yaygınlaşmaktadır.

C dili üst seviye bir programlama dili olmasına rağmen çok güçlü donanım erişimi sağlamasından dolayı gömülü sistemlerde yaygın olarak kullanılmaktadır. Dile ait ANSI ve ISO standartlarının yayınlanmasından sonra kod taşınabilirliği önemli ölçüde kolaylaşmıştır. Bu sayede yazdığınız kodu çok az değiştirerek farklı işlemci mimarileri için derlemeniz ve çalıştırmanız mümkündür. Uygulama örneği olarak UNIX, Linux işletim sistemlerinin çekirdekleri C dili ile yazılmıştır.

1.2.1 Kod Bölümleri

Her programlama dilinde olduğu gibi C dilinde de uyulması gereken bir format ve kodlama terimleri mevcuttur. Burada kısaca C dilinde kullanılan bölümleri ve etkilerini anlatmaya çalışacağım. Aşağıdaki örnek koda bakalım:

```
#include <MIKRODENETLEYICI_MODEL.h>
#include <stdio.h>
#include "modul1.h"
#include "bu_modul.h"

#define FLAG_SAYAC_DEGERI  (500)
#define DURUM_FLAG_MASKE   ((1 << 4) - 1)

static volatile unsigned int flag;

void timer_ISR (void)
{
   static unsigned int sayac;
   //....
   if (0 == sayac)
   {
      flag = 1;
      sayac = FLAG_SAYAC_DEGERI;
   }
   else
```

(continues on next page)

(önceki sayfadan devam)

```
    {
        sayac--;
    }

    //....
}

void main (void)
{
    unsigned int durum;
    //....
    //....
    if (flag == 1)
    {
        const unsigned int aktif_durum = durum & DURUM_FLAG_MASKE;
        flag = 0;
        //....
        //....
    }
    //....
    //....
}
```

1.2.2 C Programlama Dili Konuları

Uyarı: C Programlama Dili Konuları C dilini anlatmayı amaçlamamaktadır. Asıl amacı C diline ait terimlerin gömülü sistemlerdeki karşılığını ve kullanım yöntemlerini açıklamaya çalışmaktır. C diline ait açıklanmamış konular ve terimler için lütfen C programlama dilini anlatan bir kaynağa başvurun.

C Dilinde Kullanılan Bileşenler

C ile yazılan bir programın derlenme ve oluşturulma aşamalarına kısaca bakalım.

Ön işlemci (Preprocessor)

Preprocessor programın derlenmesinden önce çalıştırılan bir prosestir. Aslında bir nevi ön derleyici de denebilir. Preprocessor komutları diyez işareti (#) ile başlar. Preprocessor komutları kodun taşınabilirliğine büyük katkı sağlar. Yazmış olduğunuz kodları farklı mikrodenetleyicilerde kullanmak istiyorsanız farklı donanım ve platformlara özgü tanımlamaları tanımlamanız gerekmektedir.

```
#if defined(__ARMCC_VERSION)
const char mimari[] = "ARM"
#elif defined(__AVR__)
const char mimari[] = "AVR"
#else
const char mimari[] = "Bilinmeyen"
#endif

printf("%s\n", mimari);
```

Bu örnekte kullandığımız işlemci mimarisinin ismini printf ile yazdırabiliriz . Eğer bu kod ARM derleyici ile derlenir ise preprocessor öntanımlı `__ARMCC_VERSION` tanımından dolayı `mimari` katar dizisine `ARM` yazacaktır. Eğer bu kod parçağını `ARM` ya da `AVR` harici bir derleyici ile derlerseniz `mimari` katar dizisine `Bilinmeyen` yazacaktır.

```
#define LED_PIN  (13)

pinMode(LED_PIN, OUTPUT);
```

Bu örnekte ise preprocessor çalıştıktan sonra kod `pinMode(13, OUTPUT);` olarak değişecek ve derleyici değişmiş kod üzerinden derleme yapacaktır.

Derleyici (Compiler)

Derleyici yazmış olduğumuz C kodlarından işlemci kodlarını üreten bir yazılımdır. Eğer kod optimizasyonu aktif ise optimizasyon bu aşamada gerçekleştirilir. Her bir modül tek tek derlenerek her modüle ait obje dosyaları oluşturulur.

Bağlayıcı (Linker)

Wikipedia linker için bağlayıcı terimini kullanmış. Bu aşama son aşamadır. Oluşturulan obje dosyalarından çalışabilir uygulama oluşturulur. Bağlayıcı programa adres haritası verilmesi gerekir. Adres haritasında hafıza blokları ve özellikleri belirtilir. Aşağıda STM32F103X6 mikrodenetleyicisine ait RAM ve ROM bloklarının tanımını görebilirsiniz. Aynı script dosyasında oluştulan obje dosyalarındaki blokların nerelere yerleştireleceği, sıralaması vb. bilgiler de bulunur. Her mikroişlemci mimarisinin başlangıc adres ve yerleşim bilgileri farklıdır. Bunlar genel olarak kullanılan geliştirme ortamları (IDE) veya derleyici kütüphanelerinde üreticiler tarafından hazırlanır.

```
/*
 * Take a look in the "The GNU linker" manual, here you get
 * the following information about the "MEMORY":
 *
 * "The MEMORY command describes the location and size of
 * blocks of memory in the target."
 */
MEMORY
{
  FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 0x00008000
  RAM (rw)   : ORIGIN = 0x20000000, LENGTH = 0x00002800
}
```

Bağlayıcı; derleyici tarafından oluşturulan obje dosyalarında tanımlanmış fonksiyon, değişken, vb diğer tanımları alır ve sırayla kod bloğuna yerleştirir. Fonksiyonların, değişkenlerin adresleri bu aşamada belirlenir. En son çalışmaya hazır olan program dosyası üretilir. Bu dosya direk işlemciye yazılacak kodu, hata ayıklama bilgileri, adres haritası vb. bir çok bilgiyi de içerir. İşlemciye kopyalanacak dosya obje kopyalama programları ile bu dosya içerisinden çıkartılabilir.

C Dilinde Hafıza Yönetimi

Assembler dili ile kod yazmış olanlar RAM ve ROM yönetimine gayet aşinadır. Her bir değişkene ait adresi, sabit değerlerin adreslerini ve bunların yerleşimlerini ayarlamanız gerekmektedir. Kullanılan işlemci mimarisine göre reset ve kesme vektörlerini, özel adresleri (Bir örnek: NXP işlemcilerde kod koruma için gereken şablon değeri kodun bulunduğu ROM üzerine kaydedilmelidir . LPC176x serisi için bu adres

0x000002FC'dir) de ayarlamanız gerekmektedir. Genelde işlemci üreticileri mimariye ait yerleşim dosyalarını sağlasa dahi kendi kodunuzda kullandığınız değişken ve sabit değerleri ayarlamak zorundasınız.

Bütün üst seviye dillerde olduğu gibi C dilinde de hafıza yerleşimi ve adres yönetimi genel olarak dil bileşenleri tarafından yönetilir. C dilinde ihtiyacımız olan değişken ve büyüklüğünü belirtmemiz çoğu durumda yeterlidir.

Hafıza Elemanları

Mikrodenetleyicilerde hafıza en basit hali ile RAM, ROM ve donanım adres bloğundan oluşur. Genel olarak ROM üzerinde çalışan program, RAM üzerinde dinamik saklanan verilerimiz ve donanım adres bloğunda mikrodenetleyicinin dış dünya ile etkileşimini sağlayan donanımlar bulunur. Bu bölümde ROM ve RAM hafıza yönetimine ait kısım ile ilgileneceğiz. **Donanım** bloğuna ait tanımlamaları daha sonra inceleyeceğiz.

C Diline Ait Hafıza Terimleri

Bu bölümde C dilinin hafıza yönetiminde kullandığı terimleri çok fazla detaya inmeden anlatmaya çalışacağım.

- **STACK:** Türkçe'ye yığın/yığıntı olarak çevirilmiştir. Stack son giren ilk çıkar(LIFO) şeklinde çalışan bir depolama alanıdır. Stack fonksiyon girişlerinde ALU register(lar)ını, geri dönüş adresini, ve fonksiyonların lokal değişkenlerinin saklandığı RAM alanıdır. Bizim en çok dikkat etmemiz gereken fonksiyon lokal değişkenlerinin burada saklanıyor olmasıdır. Aşağıdaki şekilde Stack başlangıcının RAM'in en yüksek adresi olduğunu görüyoruz. Bu işlemci mimarisi ile de alakalıdır. Çoğu mimaride Stack adresi işlemci tarafından yönetilebilir. Mesela `ARM Cortex` mimarisinde R13 registeri stack pointer(SP) olarak çalışır ve aşağı yönlü büyür. Aşağıdaki şekilde gösterilen hafıza yerleşimi `ARM Cortex` mimarisine uyumludur.

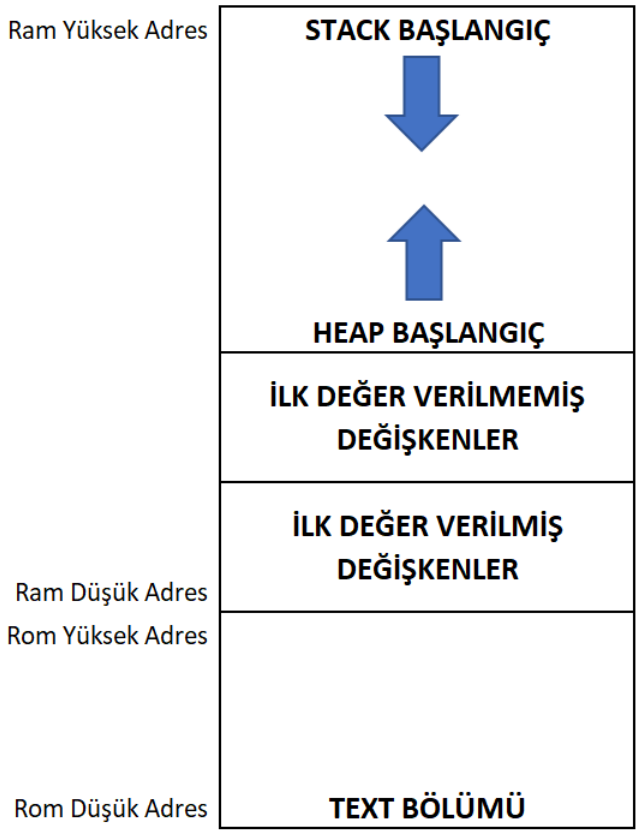
- **HEAP:** Türkçe'ye yığın/alt yığın olarak çevirilmiştir. HEAP program tarafından çalışma esnasında (run-time) dinamik olarak ihtiyaç duyulan değişkenlerin sağlandığı bloktur. Statik tanımlanmış değişkenlerden hemen sonra başlar ve yukarı yönlü büyür. `malloc`, `printf`, `sprintf` vb. tüm runtime da çalışan fonksiyonlara ait dinamik RAM istekleri buradan karşılanır. Gömülü sistemlerde bu bölge kullanıcı kodlarında mümkün olduğunca kullanılmaz. Özellikle malloc ve free fonksiyonları ile sürekli RAM bloğu alıp verme sonucunda bellek parçalanmasına sebep olur.(İng.memory fragmentation) Bunun sonucunda `HEAP` alanı `STACK` alanına taşar ise sistemin kararsız hale gelmesine yol açabilir. `HEAP` alanı bellek parçalanmasından dolayı yetersiz kalır ise programın ihtiyacı olan RAM ihtiyacı karşılanamadığı için programda istenmeyen sonuçlar oluşabilir.

- **TEXT:** Text bölümü kodun ve sadece okunabilir bilgilerin saklandığı alandır. Mikrodenetleyicilerde bu bölümde reset ve kesme vektörleri de bulunur. Bu kısım aynı zamanda ilk değer verilmiş değişkenlerin değerlerini de saklar.

- **ILK DEGER VERILMIS DEGISKENLER:** Bootstrap sırasında sadece okunabilir hafızada saklanan veriler bu alana kopyalanarak programın düzgün bir başlangıç ile başlamasını sağlar. Bu alanın büyüklüğü direk olarak ROM üzerine yazılacak imaj dosyasının büyüklüğünü etkileyecektir.

- **ILK DEGER VERILMEMIS DEGISKENLER:** Bu alan RAMde tanımlanmış değişkenlerin tutulduğu alandır. Bu alanın ilk değeri kod tarafından tanımlanmaz. Bazı kodlar bu alanı direk 0 değeri

ile doldururken bazı kodlar mikrodenetleyicinin reset anındaki rastgele değeri ne ise o şekilde bırakır. Seri porttan gelecek dataların saklandığı bir tampon alanını sıfırlamaya genelde gerek olmaz çünkü alıma başladıktan sonra gelen verileri bu alandaki verinin üzerine yazılacak ve ilk değerin bir önemi yoktur.



C Hafıza Yerleşimi

C Dilinde Kullanılan Bazı Terimler

Burada C dilinde kullanılan bazı terimlerin nasıl kullanıldığını anlatmaya çalışacağım.

#include Preprocessor

`#include` terimi başka bir başlık dosyasının içeriğinin kaynak veya başlık dosyasına dahil edilmesini sağlar. `#include` terimi ile sadece başlık dosyalarını dahil etmeniz önerilir.

Bir başlık dosyasını `#include` direktifi ile dahil ettiğinizde, o başlık dosyasındaki tüm deyim ve tanımlar dosyanıza dahil edilecektir. Bir başlık dosyası başka bir başlık dosyasını dahil edebilir. Burada önemli olan nokta içiçe başlık dosyalarını dahil etmemektir.

Elimizde 4 adet modul1, modul2, modul3 ve modul4 olarak adlandırılımış modüller olsun. Aşağıdaki şartlarda çalışması istenen bir projede başlık dosyalarını başlık dosyaları içinde dahil ederseniz bu yapıyı kuramazsınız. Başlık dosyaları içinde diğer başlık dosyalarını dahil ederseniz tüm modüller birbirine erişebilir durumda olacaktır ve hata ayıklamak zor olacaktır. Ayrıca derleme süreniz de uzayacaktır.

- Modul1; modul2'ye ve modul3'e erişebilsin, modul4'e erişemesin.

- Modul2; modul4'e erişşin, diğerlerine erişemesin
- Modul3; modul4'e erişsin, diğerlerine erişemesin

Ben kaynak dosyalarında şu sıra ile `#include` direktiflerini kullanırım.

1. mikrodenetleyiciye ait başlık dosyaları
2. Standard C kütüphaneleri başlık dosyaları
3. Dahil edilecek diğer modül başlık dosyaları
4. Kaynak dosyasına ait başlık dosyası

#define Preprocessor

`#define` terimi ile kodda kullanılacak koda özel tanımlamaları tanımlarız. Bu terim ile tek bir değer tanımlamaktan makro fonksiyon tanımlamaya kadar bir çok tanımlama yapabiliriz. Aynı zamanda farklı derleme modları için tanımlama yaparak farklı çıktı üretebiliriz.

Örnek olarak bir projede iki adet baskı devre kartı yada sistem kurdunuz. Bir tanesinde LCD ekran ile direk bilgi alırken diğer kartta LCD ekran bulunmuyor. LCD ekran üzerinde hata ayıklama mesajları gösterirken aynı kodun LCD ekran olmayan test kartında da çalışmasını istiyorsunuz. Bunu yapmak için aşağıdaki gibi bir kod yazabiliriz.

Bu kod ile `LCD_DEBUG` tanımlamasını aktif ettiğinizde sonucunuz lcd ekrana yazdırılırken, tanımlamayı deaktif ederseniz lcd ekrandan çıktı almazsınız.

volatile

C dilinde anlaşılması zor olan terimlerden birisidir `volatile` terimi. Direk çevirisi uçucu olan bu terim bir çok hata(İng.bug)'nın ana sebebidir. Kod normal çalışırken optimizasyon açıldığı anda düzgün çalışmamaya başlar ise büyük ihtimal bir değişken `volatile` tanımlanması gerekirken `volatile` olarak tanımlanmamıştır.

`volatile` değişken tanımlanırken kullanılan bir terimdir. Bu terim derleyiciye tanımlanan değişkenin uçucu bir değere sahip olduğunu belirtir.

Peki nedir bu uçucu tanımı? Uçucu; değişkenin değerinin her an değişebileceği ve her seferinde yeni değerin kullanılması gerektiğini belirtir. Aşağıdaki kod üzerinde incelersek:

```
volatile unsigned int flag;

void timer_ISR (void)
{
   //....
   flag = 1;
   //....
}

void main (void)
{
   //....
   //....
   if (flag == 1)
   {
      flag = 0;
      //....
```

(continues on next page)

(önceki sayfadan devam)

```
        //....
    }
    //....
    //....
}
```

`timer_ISR()` fonksiyonu bir zamanlama kesme fonksiyonu olarak çalışıyor. Belirli bir zaman geçtikten sonra periyodik zamanlanmış görevlerin yerine getirilmesi için `flag` değişkenine 1 değerini atıyor ve main fonksiyonu bu değeri kontrol ederek zamanlanmış görevleri yerine getiriyor. Özellikle birden fazla working register'a sahip işlemcilerde derleyiciler optimizasyon esnasında değişkenleri working registerlarda saklamaya çalışırlar ve gerekmedikçe RAM'den okuma yazma yapmazlar. Eğer derleyiciye `flag` değişkenin uçucu olduğunu belirtmez isek `main()` fonksiyonu `flag` değişkenine ait değeri working register'ına kopyalayacak ve tüm kontrolleri working registerdaki değere göre yapacaktır. `flag` değişkeninin değeri `main()` fonksiyonu haricinde bir yerde değiştirildiğinde `main()` fonksiyonu değişkeni tekrar okuyarak kontrol etmediği için yapılan değişimin bir etkisi olmayacaktır.

const

`const` **constant** kelimesinin kısaltması olarak yazılmış ve sabit anlamına gelmektedir. `const` terimi derleyiciye tanımlanan değişkenin değerinin değiştirilemeyeciğini belirtmektir. Her değiştirelemeyen değişken ROM üzerinde tanımlanmaz. Çoğu durumda `const` ile tanımlı değişkenler RAM'de tanımlıdır.

static

`static` terimini Türkçe'ye sabit yada kalıcı olarak çevirebiliriz. C dilinde `static` terimi kalıcı ve gizli görevlerini yerine getirir. Eğer bir fonksiyon içinde statik değişken tanımlarsanız bu değişken fonksiyondan çıkıldığında değerini muhafaze edecek ama o fonksiyon dışında erişilemez olacaktır. Eğer bir fonksiyon veya genel değişken tanımında `static` terimini kullanırsanız bu değişken veya fonksiyon diğer modüller tarafından ulaşılamaz olacaktır. Nesne tabanlı programlama dillerindeki **private** terimine benzer bir kullanım söz konusu olacaktır.

C Dilinde Kodlar Nasıl Derlenir?

Önceki konularda C dilinde program oluşturma aşamalarını ve hafıza yönetimini incelemiştik. Yazdığımız kodların; program oluşturma aşamasında önce *Ön İşlemci*'den geçtiğini ve *Derleyici* tarafından işlemci kodlarının oluşturulup *Bağlayıcı* ile programımızın çalışmaya hazır hale getirildiğini biliyoruz. Bu bölümde *derleyici* tarafından kodlarımızın nasıl derlendiğini, bu işlem sırasında nelere dikkat etmemiz gerektiğini inceleyeceğiz.

C dilinde yazılan her program `main()` fonksiyonuna ihtiyac duyar. `main()` fonksiyonu çağırılmadan önce yapılması gereken işlemler genellikle reset vektörüne yazılır ve işlemler bittikten sonra program `main()` fonksiyonuna koşulsuz dallanır. Eğer kodunuzda bir `main()` fonksiyonu tanımlamazsanız *Bağlayıcı* hata verecektir.

Dosya Tipleri

C dilinde kullanılan dosya tiplerini aşağıda açıklamaya çalıştım.

Kaynak Dosyası

Kaynak dosyası(İng:source file) kodlarımızın bulunduğu metin tabanlı bir dosyadır. Herhangi bir metin editörü veya geliştirme ortamı ile bu dosyaları hazırlayabiliriz. C dilinde kaynak dosyalarının uzantısı **".c"**'dir.

Başlık Dosyası

Başlık dosyası(İng:header file) kaynak dosyasında bulunan fonksiyon ve değişkenlerin diğer kaynak dosyaları tarafından kullanılabilmesini sağlar. C dilinde başlık dosyalarının uzantısı **".h"**'dir. Başlık dosyaları içinde fonksiyon prototipleri, kullanılacak tanımlar ve bazı durumlarda satıriçi(İng:inline) fonksiyonlar bulunur. Özellikle kod taşınabilirliği ve modülerlik için başlık dosyalarının düzgün hazırlanması önemlidir.

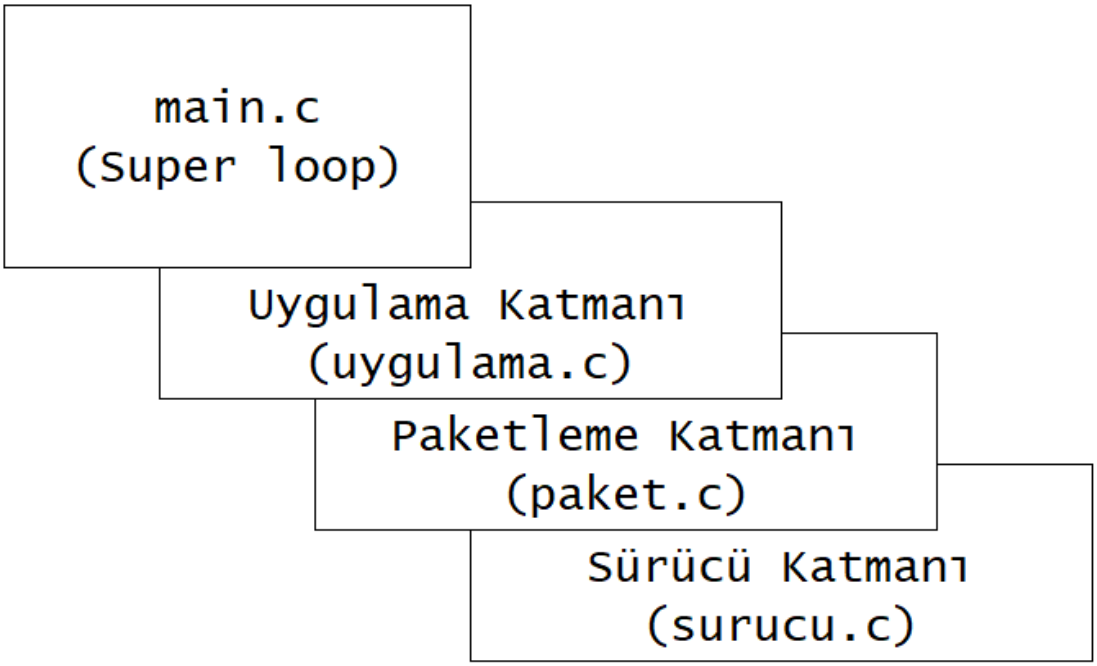
Birleştirici Dili Dosyası

Birleştirici(İng:Assembler) dili dosyası özellikle C derleyici ortamlarında başlangıç(İng:Startup) dosyası olarak kullanılır. C ve ASM kodları ayrı ayrı derlenip linker aracılığı ile program oluşturulur. ASM dosyası mikrodenetleyiciye ait açılış vektör yerleşimi, reset vektörü ve özel kodlar içerebilir.

Örnek Proje

C dilinde her assembler dosyası ve kaynak dosyası ayrı ayrı derlenerek obje dosyaları oluşturulur ve bu oluşturulan obje dosyaları linker tarafından birleştirilir. Derleme aşamasında kaynak dosyalarının içerdiği başlık dosyalarının belirli bir sıra ve düzende eklenmiş olması derleme hatalarını, modüler yapıların korunmasını, eğer katmanlı uygulama yazıyorsanız katmanlar arası ayrımı ve derlenme süresini optimize edecektir.

Aşağıdaki şekilde 3 katmanlı bir projeye ait uygulama yapısını görüyoruz. **main** modülü özellikle baremetal programlarda süper döngü(İng.Super loop) olarak kullanılır.



Örnek uygulama katmanı

Yukarıda örneğini verdiğim katmanlı uygulamalarda genelde üst katmanların bir alt katmandan daha aşağı ve alt katmanların üst katmanlara erişmesi istenmez. Bunun için içiçe eklenmiş başlık(İng.Nested-Header files) dosyalarının **kullanılmaması** gereklidir. Aşağıda her bir modül için örnek kodları bulabilirsiniz.

Main Modülü

Main modülüne ait bir başlık dosyası olmadan da kodlarımız çalışır. Bu örnek kodda main modülüne ait bir başlık dosyası kullanmayacağız. `uygulama_kontrol` değişkenine kesme fonksiyonu içinde değer ataması yapılmaktadır.

```
#include <MIKRODENETLEYICI_MODEL.h>
#include <stdbool.h>
#include "uygulama.h"

volatile unsigned int uygulama_kontrol;

void main(void)
{
   //....
   //....
   Uygulama_Init();
   //....
   //....
   while (1)
   {
      //....
      //....
      if (FALSE != uygulama_kontrol)
      {
         uygulama_kontrol = FALSE;
```

(continues on next page)

(önceki sayfadan devam)

```
            Uygulama_Kontrol();
            //....
            //....
        }
        //....
        //....
    }
}

void Timer_ISR(void)
{
    uygulama_kontrol = TRUE;
    //....
}
```

Uygulama Modülü

- **Kaynak Dosyası**

```
#include <stdio.h>
#include "paket.h"
#include "uygulama.h"

static unsigned char paket[PAKET_BOYUT_MAX];

void Uygulama_Init(void)
{
    //....
    //....
    Paket_Init();
    //....
    //....
}

void Uygulama_Kontrol(void)
{
    unsigned int uzunluk;
    //....
    //....
    uzunluk = snprintf(paket, PAKET_BOYUT_MAX, "Deneme metin\n");
    Paket_Gonder(paket, uzunluk);
}
```

- **Başlık Dosyası**

```
#ifndef UYGULAMA_H
#define UYGULAMA_H

//....
//....
void Uygulama_Init(void);
void Uygulama_Kontrol(void);
//....
//....
```

(continues on next page)

(önceki sayfadan devam)

```

#endif /* UYGULAMA_H */
```

Paket Modülü

- **Kaynak Dosyası**

```
#include <stdio.h>
#include "surucu.h"
#include "paket.h"

void Paket_Init(void)
{
   //....
   //....
   Surucu_Init();
   //....
   //....
}

void Paket_Gonder(unsigned char* veri, unsigned int uzunluk)
{
   //....
   //....
   Surucu_Gonder(veri, uzunluk);
}
```

- **Başlık Dosyası**

```
#ifndef PAKET_H
#define PAKET_H

//....
//....
#define  PAKET_BOYUT_MAX   (20)

void Paket_Init(void);
void Paket_Gonder(unsigned char*, unsigned int);
//....
//....

#endif /* PAKET_H */
```

Sürücü Modülü

- **Kaynak Dosyası**

```
#include <MIKRODENETLEYICI_MODEL.h>
#include <stdio.h>
#include "surucu.h"

void Surucu_Init(void)
{
```

(continues on next page)

(önceki sayfadan devam)

```
    //Haberleşmede kullanılacak olan mikrodenetleyici donanımına ait register
    //ayarları burada yapılır.
    //....
    //....
}

void Surucu_Gonder(unsigned char* veri, unsigned int uzunluk)
{
    //Haberleşmede kullanılan donanımın gönderme için kullanılacak register
    //atamaları burada yapılır
    //....
    //....
}
```

- **Başlık Dosyası**

```
#ifndef SURUCU_H
#define SURUCU_H

//....
//....
void Surucu_Init(void);
void Surucu_Gonder(unsigned char*, unsigned int);
//....
//....

#endif /* SURUCU_H */
```

Derleme Açıklamaları

Yukarıdaki örnek proje derlenirken hangi modülün önce derlendiğinin bir önemi yoktur ve tüm modüller kendi başlarına derlenebilir modüllerdir. Bu sayede **paketleme** katmanına ait kaynak ve başlık dosyası başka bir projeye taşınabilir ve *paketleme* katmanı sadece bir *sürücü* katmanına ihtiyaç duymaktadır. Yeni projede *sürücü* katmanı yukarıdaki standardda yazılmış ise paketleme katmanınız sorunsuz çalışacaktır.

Not: Yukarıdaki örnekte başlık dosyaları hiçbir `#include` direktifine sahip değildir. Çok gerekmedikçe ben başlık dosyaları içinde özellikle projeye ait diğer modüllerin başlık dosyalarının içerilmesini önermiyorum. Eğer *uygulama.h* dosyası içinde *paket.h* içerilse idi **main** modülünden **paket** modülü fonksiyonlarına erişebilirdiniz.

Bu tarz kontrolsüz erişimler mantık hataları yapılmasına yardımcı olur. Main modülü içinde `Uygulama_Init()` fonksiyonu çağırılmadan `Paket_Gonder()` fonksiyonu çağırılır ise istenmeyen sonuçlar ortaya çıkacaktır.

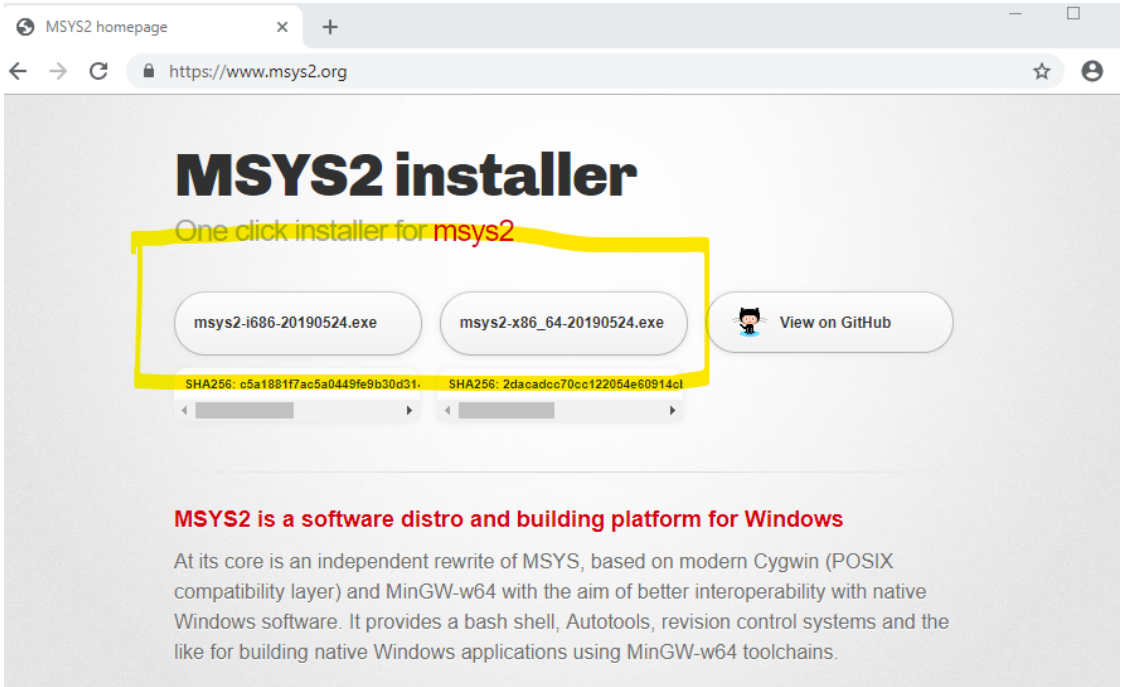
1.3 MSYS2 Kurulumu

MSYS2 platformu en sade hali ile Windows üzerinde çalışan bir Bash terminal emülatorüdür. Kurulum için gerekli olan bileşenler:

1. İnternet bağlantısı
2. MSYS2

3. İlk Kurulum Script

MSYS2 websitesinden en son paketlenen sürümünü indirip kurun. 64-bit sürümünü kurmanızı tavsiye ederim.



Kurulum bittikten sonra başlat menünüze MSYS2 klasörü gelecektir. Başlat menüsünden **MSYS2 MinGW 64-bit** olan sürümü çalıştırın.

Açılan terminal ekranına `pacman -Syu` komutunu yazıp onaylayın.

```
tester@test-w8 MINGW64 ~
$ pacman -Syu
:: Paket veritabanları senkronize ediliyor...
 mingw32                 548,0 KiB   323K/s 00:02 [#####################] 100%
 mingw32.sig             119,0   B  0,00B/s 00:00 [#####################] 100%
 mingw64                 549,2 KiB   438K/s 00:01 [#####################] 100%
 mingw64.sig             119,0   B  11,6K/s 00:00 [#####################] 100%
 msys                    180,7 KiB   561K/s 00:00 [#####################] 100%
 msys.sig                119,0   B  0,00B/s 00:00 [#####################] 100%
:: Starting core system upgrade...
uyarı: terminate other MSYS2 programs before proceeding
paket bağımlılıkları çözümleniyor...
varsa paketler arası çakışmalara bakılıyor...

Paketler (3) mintty-1~3.0.1-1  msys2-runtime-3.0.7-3  pacman-5.1.3-2

Toplam İndirme Boyutu:  17,43 MiB
Toplam Kurulum Boyutu:  60,02 MiB
Güncelleme Boyutu:      0,02 MiB

:: Kuruluma onay veriyor musunuz? [E/h] |
```

Sistem güncellemesi yapıldıktan sonra terminal penceresini direk kapatmanızı isteyen bir not göreceksiniz. Terminal ekranın sağ üst köşesindeki **X** kapatma düğmesine tıklayarak pencereyi kapatın. Tıkladığınızda bir uyarı verecektir onaylayıp pencereyi kapatın.

```
 mingw32                 548,0 KiB   323K/s 00:02 [#####################] 100%
 mingw32.sig             119,0   B  0,00B/s 00:00 [#####################] 100%
 mingw64                 549,2 KiB   438K/s 00:01 [#####################] 100%
 mingw64.sig             119,0   B  11,6K/s 00:00 [#####################] 100%
 msys                    180,7 KiB   561K/s 00:00 [#####################] 100%
 msys.sig                119,0   B  0,00B/s 00:00 [#####################] 100%
:: Starting core system upgrade...
uyarı: terminate other MSYS2 programs before proceeding
paket bağımlılıkları çözümleniyor...
varsa paketler arası çakışmalara bakılıyor...

Paketler (3) mintty-1~3.0.1-1  msys2-runtime-3.0.7-3  pacman-5.1.3-2

Toplam İndirme Boyutu:  17,
Toplam Kurulum Boyutu:  60,
Güncelleme Boyutu:      0,0

:: Kuruluma onay veriyor mu
:: Paketler alınıyor...
 msys2-runtime-3.0.7...                                [#####################] 100%
 mintty-1~3.0.1-1-x86_64                               [#####################] 100%
 pacman-5.1.3-2-x86_64     14,5 MiB  1945K/s 00:08     [#####################] 100%
(3/3) anahtarlıktaki anahtarlar kontrol ediliyor       [#####################] 100%
(3/3) paket bütünlüğü kontrol ediliyor                 [#####################] 100%
(3/3) paket dosyaları yükleniyor                       [#####################] 100%
(3/3) dosya çakışmaları kontrol ediliyor               [#####################] 100%
(3/3) disk alanı kontrol ediliyor                      [#####################] 100%
:: Paket değişiklikleri işleniyor...
(1/3) güncelleniyor msys2-runtime                      [#####################] 100%
(2/3) güncelleniyor mintty                             [#####################] 100%
(3/3) güncelleniyor pacman                             [#####################] 100%
uyarı: terminate MSYS2 without returning to shell and check for updates again
uyarı: for example close your terminal window instead of calling exit
```

```
mintty
Processes are running in session:
| WPID    PID   COMMAND
| 9900   1737   vim
Close anyway?
OK   Cancel
```

Tekrar **MSYS2 MinGW 64-bit** terminalini açın. Az önce verdiğimiz `pacman -Syu` komutunu tekrar yazıp çalıştırın. Bu sefer sistem güncellemesi değil kurulu paketlerin güncellenmesini onaylamanızı isteyecektir. Onaylayıp kurulumun bitmesini bekleyin.

```
tester@test-w8 MINGW64 ~
$ pacman -Syu
:: Paket veritabanları senkronize ediliyor...
 mingw32 deposu güncel
 mingw64 deposu güncel
 msys deposu güncel
:: Starting core system upgrade...
 yapılacak bir şey yok
:: Sistem güncellemesi başlatılıyor...
paket bağımlılıkları çözümleniyor...
varsa paketler arası çakışmalara bakılıyor...

Paketler (18) bash-completion-2.9-1  bsdcpio-3.4.0-1  bsdtar-3.4.0-1
              curl-7.65.1-1  gnupg-2.2.16-1  less-551-1  libarchive-3.4.0-1
              libcurl-7.65.1-1  libedit-20190324_3.1-1  libgnutls-3.6.8-1
              libgpgme-1.13.1-1  libiconv-1.16-1  libidn2-2.2.0-1
              libopenssl-1.1.1.c-1  libp11-kit-0.23.16.1-1  libpsl-0.21.0-1
              openssl-1.1.1.c-1  p11-kit-0.23.16.1-1

Toplam İndirme Boyutu:  12,53 MiB
Toplam Kurulum Boyutu:  88,63 MiB
Güncelleme Boyutu:      1,04 MiB

:: Kuruluma onay veriyor musunuz? [E/h]
```

Güncellemeler bittikten sonra İlk Kurulum Script dosyasını indirip MSYS2 kullanıcı kllasörüne kopyalayın. MSYS2 kullanıcı klasörü MSYS2 kurulum klasörü altında **\home\KullanıcıAdı** klasörüdür. Klasöre kopyaladıktan sonra terminalde `cd #` veya `cd ~` komutunu verin. `ls` komutu ile dizin içeriğini listeleyin. Listede **ilk_kurulum_script.sh** dosyasını gördükten sonra `./ilk_kurulum_script.sh` komutu ile kurulumu başlatın.

```
tester@test-w8 MINGW64 ~
$ cd #

tester@test-w8 MINGW64 ~
$ ls
blink  ilk_kurulum_script.sh

tester@test-w8 MINGW64 ~
$ ./ilk_kurulum_script.sh
```

Kurulum süresi bağlantı hızınız, işlemci hızını vb. etkenlere göre değişiklik gösterir ve 1-5 dk. sürebilir. Terminal ekranında durumu izleyebilirsiniz. Kurulum bittikten sonra ARM GNU toolchain bilgisayarınıza kurulmuş olacaktır.

1.4 GNU Arm Toolchain Kurulumu

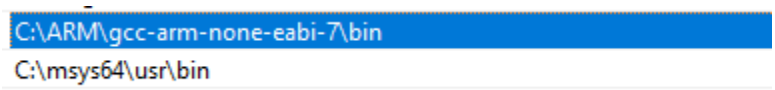
ARM çekirdeğine sahip mikrodenetleyici ve mikroişlemcilerine yazılan C/C++ kodlarını derlemek için kullanılan derleyicilerden bir tanesi de GNU ARM derleyici topluluğudur. Bu derleyici topluluğu GNU C derleyicisinin ARM firması tarafından dağıtılan bir versiyondur. ARM firması ayrıca ücretli ARM derleyicisi de sağlamaktadır. İki derleyici arasındaki farklar genelde optimizasyon odaklı konularda belli olmaktadır. Eğer ücretsiz sunulan GNU Arm derleyici performans konusunda işinize yaramaz ise diğer derleyicileri denemenizi tavsiye ederim.

GNU toolchain versiyonu olarak **ARM GNU 7.3.1 (GNU Tools for Arm Embedded Processors 7-2018-q2-update)** seçilmiştir. Bu versiyonun seçilme sebebi **Atollic**, **STM32CubeIDE**, **Atmel Studio** geliştirme ortamlarının da **7.3.1** versiyonunu kullanıyor olmasıdır.

ARM GNU Toolchain websitesinde detaylı bilgileri bulabilirsiniz.

1.4.1 Windows Ayarları

ARM GNU toolchain **MinGW** terminali altında script ile kurulduktan sonra herhangi bir ek ayara gerek kalmadan çalışacaktır. Fakat Windows için ARM GNU toolchain ve MSYS2 kurulumlarının PATH'e eklenmesi gerekmektedir. Windows ortam değişkenlerinden PATH değişkenine **C:\ARM\gcc-arm-none-eabi-7\bin** ve **X:\msys64\usr\bin** yollarını eklemeniz gereklidir. Eğer **MSYS2** kurulumunu başka bir yere yaptı iseniz yolu değiştirmeniz gerekli.



1.5 Arduino Geliştirme Ortamı

Arduino gcc-avr derleyici topluluğunun C/C++ derleyicisini kullanır. AVR ve bazı 3. parti kütüphaneler harici bir çok kütüphanesi C++ ile yazılmıştır. IDE üzerinde yazılan kodlarınız aslında bir C++ kod dosyası içine alınarak aşağıdaki main.cpp dosyası içinden çağırılır.

Not: Özellikle gömülü sistemler alanında eğitim veren mühendislik bölümlerinde **sadece** Arduino öğretilmesine karşıyım. Normalde Arduino örneği yazmayı düşünmüyordum ama matematik karşısında elim kolum bağlandı. Matematik diyor ki hiçbir şey yapmamak ile az da olsa bir şeyler ile başlayarak öğrenmek arasında sonsuz oran vardır. Öğrenmede tatmin edicilik olması ve gerçek dünyada çalışan bir kodu görmeniz için açısından ucuz ve kolay bulunan Arduino bordları için de örneklere yer vereceğim.

Uyarı: Arduino IDE'sinin hibrit yapısında dikkat etmeniz bir kaç nokta vardır. Normal C projeleri gibi bir main() fonksiyonu yoktur. Bunun yerine setup() ve loop() fonksiyonlarını sağlar. Ayrıca fonksiyonların prototip tanımlarını kodunuzu tarayarak otomatik olarak çıkartır.

- Şemalar simulasyon odaklı verilecektir. Simulasyon için Proteus yazılımı kullanılacaktır. Proteus versiyon 8.7 SP3'dür. Proteus nereden bulunur vb. soruları lütfen sormayınız.
- Arduino Atmega328p çipine sahip UNO veya NANO
- Arduino IDE

```
/*
  main.cpp - Main loop for Arduino sketches
  Copyright (c) 2005-2013 Arduino Team.  All right reserved.

  This library is free software; you can redistribute it and/or
  modify it under the terms of the GNU Lesser General Public
  License as published by the Free Software Foundation; either
  version 2.1 of the License, or (at your option) any later version.

  This library is distributed in the hope that it will be useful,
  but WITHOUT ANY WARRANTY; without even the implied warranty of
  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the GNU
  Lesser General Public License for more details.

  You should have received a copy of the GNU Lesser General Public
  License along with this library; if not, write to the Free Software
  Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA  02110-1301  USA
*/

#include <Arduino.h>

// Declared weak in Arduino.h to allow user redefinitions.
int atexit(void (* /*func*/ )()) { return 0; }

// Weak empty variant initialization function.
// May be redefined by variant files.
void initVariant() __attribute__((weak));
void initVariant() { }

void setupUSB() __attribute__((weak));
void setupUSB() { }

int main(void)
{
        init();

        initVariant();

#if defined(USBCON)
        USBDevice.attach();
#endif

        setup();

        for (;;) {
                loop();
                if (serialEventRun) serialEventRun();
        }

        return 0;
}
```

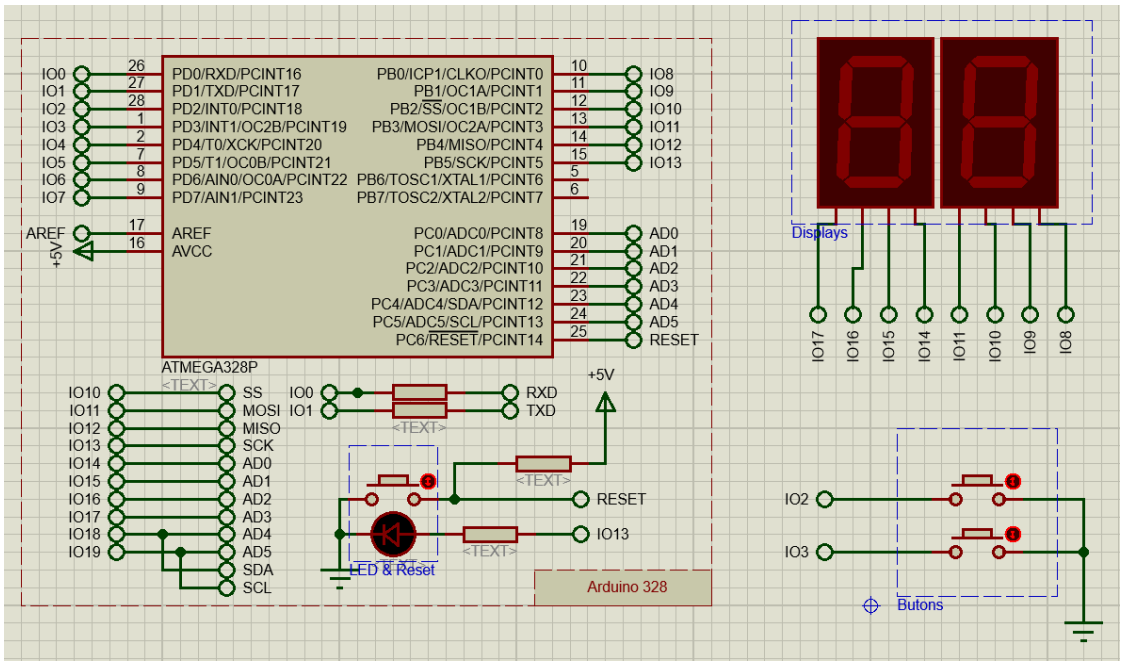
1.6 7 Segment BCD

İlk örnek olan *Aşağı Yukarı Sayıcı* örneğinde farklı şekillerde yazılmış kod örnekleri verdim. Özellikle farklı şekillerde kod yazmak direk kod yazmaktan çok daha fazla vakit alıyor. O yüzden diğer örnekler için bir örnek vermeyi planlıyorum. Çok gerekli olmadıkça diğer alıştımalarda farklı kod örnekleri yazmayacağım.

İlk örnekte yeni başlayanlara kolaylık olması için 7 segment LED display BCD -> 7 Segment Decoder (örnek 4056) entegre edilmiş direk BCD sürülebilen ekranlar seçtim.

1.6.1 Bağlantı Şeması

Aşağıdaki şemada ATmega328p mikrodenetleyicisine bağlanmış iki adet 7 segment LED display ve iki buton bulunmaktadır. Şema simulasyon için yeterli elemanlara sahiptir.



7-Segment BCD Şema

Buradan bu örneğe ait proje dosyasını indirebilirsiniz.

1.6.2 Örnek Uygulamalar

Yukarıdaki bağlantı şeması ile birlikte aşağıdaki örnekleri kullanabilirsiniz.

Aşağı Yukarı Sayıcı

Bu örnek iki buton ile 0 - 99 sayıcı örneğidir. Aşağıda aynı işi yapan farklı kod örnekleri vardır.

İlk örnek kod taşınabilirliği ve anlaşılabilirliği açısından zayıf bir örnektir. Kodlar optimize yazılmamış ve kod yoğunluğu gayet düşük bir koddur.

İkinci örnek aynı kodun taşınabilirliği, okunabilirliği ve optimize edilmiş halini içerir. Pin giriş ve çıkışların Arduino kütüphaneleri kullanılmıştır.

Üçüncü örnek herhangi bir kütüphane kullanmadan bir mikrodenetleyici nasıl C ile kodlanır örneğidir. Bu örnek ile ikinci örneği karşılaştırıp **Arduino** fonksiyonları yerine direk mikrodenetleyici registerları ile nasıl yapıldığını anlamaya çalışmanızı tavsiye ederim.

Not: Üçüncü örnek de Arduino IDE'sinde yapılacağı için **setup()**, **loop()** fonksiyonları bu örnekte de bulunmaktadır.

Örnek - 1

Bu örnek ilk programlamaya başladığım zamanlarda yazdığım kodlara çok benzer :) Listede bu kodun eksik noktaları belirtilmiştir.

1. Tekrarlanan işler döngülere alınmamış Kopyala-yapıştır ile çoğaltılmış
2. Tanımlama hiç kullanılmamış, en ufak pin değişikliğinde tüm kodun elden geçmesi gerekir
3. Aynı işi yapan iki ayrı fonksiyon kullanılmış
4. Lokal bir değişken ile çözülebilecek `sayac` değişkeni global olarak tanımlanmış
5. Kod yoğunluğu ve derleyici optimizasyonu uygunluğu düşük bir kod

```
static unsigned int sayac;

void setup()
{
   unsigned int pin;

   pinMode(2, INPUT_PULLUP);
   pinMode(3, INPUT_PULLUP);
   pinMode(13, OUTPUT);
   pinMode(8, OUTPUT);
   pinMode(9, OUTPUT);
   pinMode(10, OUTPUT);
   pinMode(11, OUTPUT);

   pinMode(14, OUTPUT);
   pinMode(15, OUTPUT);
   pinMode(16, OUTPUT);
   pinMode(17, OUTPUT);

   sayac = 0;
}

void loop()
{
   if (LOW == digitalRead(3))
   {
      digitalWrite(13, HIGH);
      while (LOW == digitalRead(3))
      {
         /* Buton basılı iken bekle */
      }
      digitalWrite(13, LOW);
      if (0 != sayac)
      {
```

(continues on next page)

(önceki sayfadan devam)

```
            sayac--;
         }
            else
         {
            sayac = 0;
         }
      }
      if (LOW == digitalRead(2))
      {
         digitalWrite(13, HIGH);
         while (LOW == digitalRead(2))
         {
            /* Buton basılı iken bekle */
         }
         digitalWrite(13, LOW);
         if (sayac < 99)
         {
            sayac++;
         }
         else
         {
            sayac=99;
         }
      }

      Onlar_Yazdir();
      Birler_Yazdir();
}

void Onlar_Yazdir(void)
{
      unsigned int onlar;

      onlar = sayac / 10;

      if (onlar & 1)
      {
         digitalWrite(14, HIGH);
      }
      else
      {
         digitalWrite(14, LOW);
      }
      if (onlar & 2)
      {
         digitalWrite(15, HIGH);
      }
      else
      {
         digitalWrite(15, LOW);
      }
      if (onlar & 4)
      {
         digitalWrite(16, HIGH);
      }
      else
      {
```

(continues on next page)

(önceki sayfadan devam)

```
        digitalWrite(16, LOW);
    }
    if (onlar & 8)
    {
        digitalWrite(17, HIGH);
    }
    else
    {
        digitalWrite(17, LOW);
    }
}

void Birler_Yazdir(void)
{
    unsigned int birler;

    birler = sayac % 10;

    if (birler & 1)
    {
        digitalWrite(8, HIGH);
    }
    else
    {
        digitalWrite(8, LOW);
    }
    if (birler & 2)
    {
        digitalWrite(9, HIGH);
    }
    else
    {
        digitalWrite(9, LOW);
    }
    if (birler & 4)
    {
        digitalWrite(10, HIGH);
    }
    else
    {
        digitalWrite(10, LOW);
    }
    if (birler & 8)
    {
        digitalWrite(11, HIGH);
    }
    else
    {
        digitalWrite(11, LOW);
    }
}
```

Örnek - 2

Bu örnekte C dilinde ön-işlemci tanımlamaları, döngüler, bit işlemleri, **? : operatörü**, fonksiyonlaştırma örneklerini göstermeye çalıştım. LED Display bağlantıları değişirse sadece ilk baştaki tanımları değiştirmeniz yeterlidir. Hatta maksimum değer, sayı tabanı vb. değerleri değiştirdiğinizde sonuç çok kolay bir şekilde üretilecektir. Üçüncü bir display ekleyip 8'li sayı tabanında bir sayıcı yapmayı deneyebilirsiniz.

```
#define BIRLER_BASAMAGI_BASLANGIC    (8U)
#define ONLAR_BASAMAGI_BASLANGIC    (14U)
#define DISPLAY_PIN_SAYISI           (4U)
#define YUKARI_DUGME                 (2U)
#define ASAGI_DUGME                  (3U)
#define LED_PIN                     (13U)
#define SAYAC_MAKSIMUM              (99U)
#define SAYI_TABANI                 (10U)

void setup()
{
   unsigned int i;

   pinMode(YUKARI_DUGME, INPUT_PULLUP);
   pinMode(ASAGI_DUGME, INPUT_PULLUP);
   pinMode(LED_PIN, OUTPUT);
   for (i = 0; i < DISPLAY_PIN_SAYISI; i++)
   {
      pinMode((BIRLER_BASAMAGI_BASLANGIC + i), OUTPUT);
      pinMode((ONLAR_BASAMAGI_BASLANGIC + i), OUTPUT);
   }
}

void loop()
{
   static unsigned int sayac = 0;

   if (LOW == digitalRead(ASAGI_DUGME))
   {
      digitalWrite(LED_PIN, HIGH);
      while (LOW == digitalRead(ASAGI_DUGME))
      {
         /* Buton basılı iken bekle */
      }
      digitalWrite(LED_PIN, LOW);
      sayac--;
      if (SAYAC_MAKSIMUM < sayac)
      {
         sayac = 0;
      }
   }
   if (LOW == digitalRead(YUKARI_DUGME))
   {
      digitalWrite(LED_PIN, HIGH);
      while (LOW == digitalRead(YUKARI_DUGME))
      {
         /* Buton basılı iken bekle */
      }
      digitalWrite(LED_PIN, LOW);
      sayac++;
```

(continues on next page)

(önceki sayfadan devam)

```
51        if (sayac >= SAYAC_MAKSIMUM)
52        {
53           sayac = SAYAC_MAKSIMUM;
54        }
55     }
56     Yazdir(BIRLER_BASAMAGI_BASLANGIC, (sayac % SAYI_TABANI));
57     Yazdir(ONLAR_BASAMAGI_BASLANGIC,  (sayac / SAYI_TABANI));
58  }
59
60  void Yazdir (unsigned int ondalik, unsigned int sayi)
61  {
62     unsigned int i;
63     for (i = 0; i < DISPLAY_PIN_SAYISI; i++)
64     {
65        digitalWrite(ondalik++, (sayi & (1U << i)) ? HIGH : LOW);
66     }
67  }
```

Örnek - 3

Bu örnekte ise mikrodenetleyiciye ait bir sürücü kütüphanemiz yok. Tüm giriş ve çıkış işlemleri mikrodenetleyici Port donanımı üzerinden yapılacak. Sadece AVR derleyici ve üretici tanım başlık dosyaları kullanılmıştır. PORTB adresini elle tanımlamak yerine üretici başlık dosyasındaki tanımı kullanır.

```
1  #define BIRLER_BASAMAGI_MASKE          (0x0F)
2  #define ONLAR_BASAMAGI_MASKE           (0x0F)
3  #define BIRLER_BASAMAGI_YON_REGISTER   DDRB
4  #define ONLAR_BASAMAGI_YON_REGISTER    DDRC
5  #define BIRLER_BASAMAGI_PORT_REGISTER  PORTB
6  #define ONLAR_BASAMAGI_PORT_REGISTER   PORTC
7  #define DUGMELER_YON_REGISTER          DDRD
8  #define DUGMELER_PORT_REGISTER         PORTD
9  #define DUGMELER_PIN_REGISTER          PIND
10 #define YUKARI_DUGME                   (1 << 2U)
11 #define ASAGI_DUGME                    (1 << 3U)
12 #define LED_PORT_REGISTER              PORTB
13 #define LED_PIN                        (1 << 5U)
14 #define SAYAC_MAKSIMUM                 (99U)
15 #define SAYI_TABANI                    (10U)
16
17 void setup()
18 {
19    BIRLER_BASAMAGI_YON_REGISTER = (BIRLER_BASAMAGI_MASKE | LED_PIN);
20    ONLAR_BASAMAGI_YON_REGISTER  = ONLAR_BASAMAGI_MASKE;
21    DUGMELER_YON_REGISTER        = ~(YUKARI_DUGME | ASAGI_DUGME);
22    DUGMELER_PORT_REGISTER       = (YUKARI_DUGME | ASAGI_DUGME);
23 }
24
25 void loop()
26 {
27    static unsigned int sayac = 0;
28
29    if (ASAGI_DUGME != (ASAGI_DUGME & DUGMELER_PIN_REGISTER))
30    {
31       LED_PORT_REGISTER |= LED_PIN;
```

(continues on next page)

(önceki sayfadan devam)

```
32        while (ASAGI_DUGME != (ASAGI_DUGME & DUGMELER_PIN_REGISTER))
33        {
34           /* Buton basılı iken bekle */
35        }
36        LED_PORT_REGISTER &= ~LED_PIN;
37        sayac--;
38        if (SAYAC_MAKSIMUM < sayac)
39        {
40           sayac = 0;
41        }
42     }
43     if (YUKARI_DUGME != (YUKARI_DUGME & DUGMELER_PIN_REGISTER))
44     {
45        LED_PORT_REGISTER |= LED_PIN;
46        while (YUKARI_DUGME != (YUKARI_DUGME & DUGMELER_PIN_REGISTER))
47        {
48           /* Buton basılı iken bekle */
49        }
50        LED_PORT_REGISTER &= ~LED_PIN;
51        sayac++;
52        if (sayac >= SAYAC_MAKSIMUM)
53        {
54           sayac = SAYAC_MAKSIMUM;
55        }
56     }
57     Yazdir(&BIRLER_BASAMAGI_PORT_REGISTER, (sayac % SAYI_TABANI), BIRLER_
↪BASAMAGI_MASKE);
58     Yazdir(&ONLAR_BASAMAGI_PORT_REGISTER, (sayac / SAYI_TABANI), ONLAR_
↪BASAMAGI_MASKE);
59  }
60
61  void Yazdir(volatile uint8_t *const port, const uint8_t deger, const uint8_t␣
↪maske)
62  {
63     uint8_t port_gecici_deger;
64
65     port_gecici_deger  = *port & ~maske;
66     port_gecici_deger |= deger & maske;
67     *port = port_gecici_deger;
68  }
```

Aşağı Yukarı Otomatik Sayıcı

Bir önceki örnekte arttırma ve azaltma için sürekli butona basıp bırakmak gerekiyor. Bu örnekte buton basılı tutulduğunda arttırma ve azaltmanın otomatik olarak nasıl arttırılabileceği ile ilgili bir örnek vereceğim. Bu örnek birkaç tane bug içermektedir. Bugları ve çözümünü bulmak size kalmış :)

```
1  #define BIRLER_BASAMAGI_BASLANGIC    (8U)
2  #define ONLAR_BASAMAGI_BASLANGIC    (14U)
3  #define DISPLAY_PIN_SAYISI           (4U)
4  #define YUKARI_DUGME                 (2U)
5  #define ASAGI_DUGME                  (3U)
6  #define LED_PIN                     (13U)
7  #define SAYAC_MAKSIMUM              (99U)
8  #define SAYI_TABANI                 (10U)
```

(continues on next page)

(önceki sayfadan devam)

```
#define DEGISIM_ZAMAN_FARKI_MS      (500U)

void setup()
{
   unsigned int i;

   pinMode(YUKARI_DUGME, INPUT_PULLUP);
   pinMode(ASAGI_DUGME, INPUT_PULLUP);
   pinMode(LED_PIN, OUTPUT);
   for (i = 0; i < DISPLAY_PIN_SAYISI; i++)
   {
      pinMode((BIRLER_BASAMAGI_BASLANGIC + i), OUTPUT);
      pinMode((ONLAR_BASAMAGI_BASLANGIC + i), OUTPUT);
   }
}

void loop()
{
   static unsigned int sayac = 0;
   static unsigned char asagi_basili;
   static unsigned long asagi_artis_bekleme;
   static unsigned char yukari_basili;
   static unsigned long yukari_artis_bekleme;
   const unsigned long yeni_zaman = millis();

   if (LOW == digitalRead(ASAGI_DUGME))
   {
      if (false != asagi_basili)
      {
         asagi_basili = true;
         sayac--;
         asagi_artis_bekleme = yeni_zaman + DEGISIM_ZAMAN_FARKI_MS;
      }
      else
      {
         if (yeni_zaman >= asagi_artis_bekleme)
         {
            sayac--;
            asagi_artis_bekleme = yeni_zaman + DEGISIM_ZAMAN_FARKI_MS;
         }
      }
      if (SAYAC_MAKSIMUM < sayac)
      {
         sayac = 0;
      }
   }
   else
   {
      asagi_basili = false;
   }

   if (LOW == digitalRead(YUKARI_DUGME))
   {
      if (false != yukari_basili)
      {
         yukari_basili = true;
         sayac++;
```

(continues on next page)

(önceki sayfadan devam)

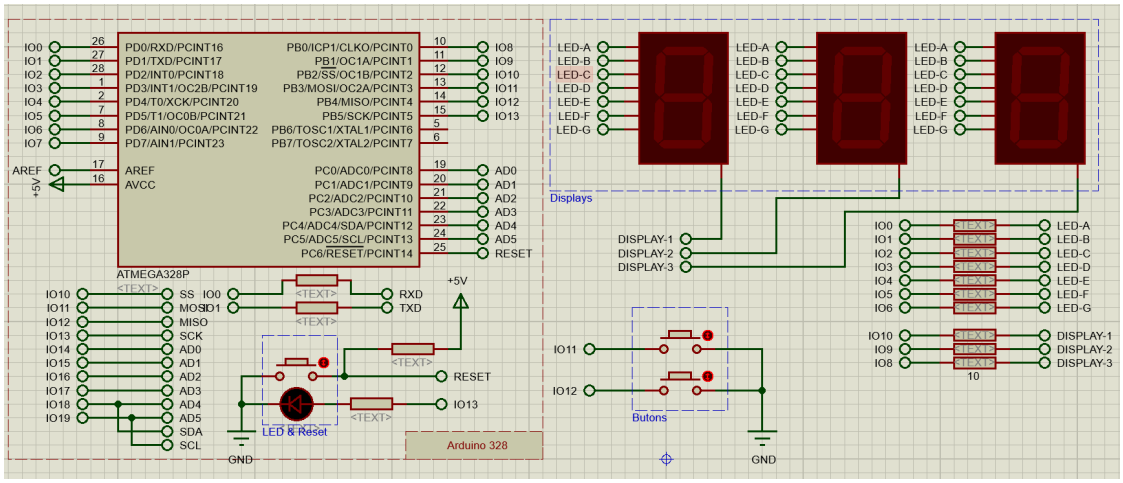
```
            yukari_artis_bekleme = yeni_zaman + DEGISIM_ZAMAN_FARKI_MS;
         }
         else
         {
            if (yeni_zaman >= yukari_artis_bekleme)
            {
               sayac++;
               yukari_artis_bekleme = yeni_zaman + DEGISIM_ZAMAN_FARKI_MS;
            }
         }
         if (sayac >= SAYAC_MAKSIMUM)
         {
            sayac = SAYAC_MAKSIMUM;
         }
      }
      else
      {
         yukari_basili = false;
      }

      Yazdir(BIRLER_BASAMAGI_BASLANGIC, (sayac % SAYI_TABANI));
      Yazdir(ONLAR_BASAMAGI_BASLANGIC,  (sayac / SAYI_TABANI));
}

void Yazdir (unsigned int ondalik, unsigned int sayi)
{
   unsigned int i;
   for (i = 0; i < DISPLAY_PIN_SAYISI; i++)
   {
      digitalWrite(ondalik++, (sayi & (1U << i)) ? HIGH : LOW);
   }
}
```

1.7 7 Segment Tarama

7 Segment tarama projesinde BCD decoder bağlanmış displayler yerine 8 girişli 7 segment displayler kullanılmıştır. Basamak sayısı 3'e çıkartılmış ve displaylerdeki değerler sabit çıkış yerine sürekli tarama yapılarak gösterilmiştir.

1.7.1 Bağlantı Şeması

Aşağıdaki şemada ATmega328p mikrodenetleyicisine bağlanmış üç adet 7 segment LED display ve iki buton bulunmaktadır. Şemada kullanılan bütün dirençler **Proteus**'da digital olarak seçilmiştir. Bu özellik ile digital bağlantılarda kullanılan dirençler simulasyon esnasında analog hesaplamalara tabi tutulmaz. Bu sayede simulasyon daha hızlı çalışır.



7-Segment Tarama Şeması

Buradan bu örneğe ait proje dosyasını indirebilirsiniz.

1.7.2 Örnek Uygulamalar

Yukarıdaki bağlantı şeması ile birlikte aşağıdaki örnekleri kullanabilirsiniz.

Aşağı Yukarı Sayıcı

Bu örnek iki buton ile 0 - 999 sayıcı örneğidir. *7 segment BCD Örnek-2*'den otomatik artırım kısmını miras almıştır.

```
#define DISPLAY_DATA_YON_REGISTER          DDRD
#define DISPLAY_DATA_REGISTER              PORTD
#define DISPLAY_CONTROL_YON_REGISTER       DDRB
#define DISPLAY_CONTROL_DATA_REGISTER      PORTB

#define LED_PIN                      (13U)
#define DISPLAY_DATA_MASKE          (0X7F)
#define DISPLAY_CONTROL_MASKE       (0X07)

#define DISPLAY_SAYISI                (3U)

#define YUKARI_DUGME                 (11U)
#define ASAGI_DUGME                  (12U)

#define SAYAC_MAKSIMUM              (999U)
#define SAYI_TABANI                  (10U)
#define DEGISIM_ZAMAN_FARKI_MS      (500U)
#define DISPLAY_BEKLEME_US         (1750U)

static const unsigned char numaralar[] = {
//    0     1     2     3     4     5     6     7     8     9
   0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f
};

void setup()
{
```

(continues on next page)

(önceki sayfadan devam)

```
   DISPLAY_CONTROL_DATA_REGISTER = 6;
   pinMode(YUKARI_DUGME, INPUT_PULLUP);
   pinMode(ASAGI_DUGME, INPUT_PULLUP);
   pinMode(LED_PIN, OUTPUT);
   DISPLAY_DATA_YON_REGISTER = DISPLAY_DATA_MASKE;
   DISPLAY_DATA_REGISTER     = numaralar[0];
   DISPLAY_CONTROL_YON_REGISTER  = DISPLAY_CONTROL_MASKE;
}

void loop()
{
   static unsigned int sayac = 0;
   static unsigned char asagi_basili;
   static unsigned char yukari_basili;
   static unsigned long asagi_artis_bekleme;
   static unsigned long yukari_artis_bekleme;
   const unsigned long yeni_zaman = millis();

   if (LOW == digitalRead(ASAGI_DUGME))
   {
      if (false != asagi_basili)
      {
         asagi_basili = true;
         sayac--;
         asagi_artis_bekleme = yeni_zaman + DEGISIM_ZAMAN_FARKI_MS;
      }
      else
      {
         if (yeni_zaman >= asagi_artis_bekleme)
         {
            sayac--;
            asagi_artis_bekleme = yeni_zaman + DEGISIM_ZAMAN_FARKI_MS;
         }
      }
      if (SAYAC_MAKSIMUM < sayac)
      {
         sayac = 0;
      }
   }
   else
   {
      asagi_basili = false;
   }
   if (LOW == digitalRead(YUKARI_DUGME))
   {
      if (false != yukari_basili)
      {
         yukari_basili = true;
         sayac++;
         yukari_artis_bekleme = yeni_zaman + DEGISIM_ZAMAN_FARKI_MS;
      }
      else
      {
         if (yeni_zaman >= yukari_artis_bekleme)
         {
            sayac++;
            yukari_artis_bekleme = yeni_zaman + DEGISIM_ZAMAN_FARKI_MS;
```

(continues on next page)

(önceki sayfadan devam)

```
            }
         }
         if (sayac >= SAYAC_MAKSIMUM)
         {
            sayac = SAYAC_MAKSIMUM;
         }
      }
      else
      {
         yukari_basili = false;
      }
      Yazdir(sayac);
   }

   void Yazdir (const unsigned int sayi)
   {
      unsigned int i;
      unsigned char display;
      unsigned int numara;
      unsigned int sayi_ic = sayi;

      for (i = 0; i < DISPLAY_SAYISI; i++)
      {
         display = (~(1U << i)) & DISPLAY_CONTROL_MASKE;
         numara = sayi_ic % SAYI_TABANI;
         DISPLAY_CONTROL_DATA_REGISTER = (DISPLAY_CONTROL_DATA_REGISTER & ~
↪DISPLAY_CONTROL_MASKE) | display;
         DISPLAY_DATA_REGISTER = numaralar[numara];
         sayi_ic /= SAYI_TABANI;
         delayMicroseconds(DISPLAY_BEKLEME_US);
      }
   }
```

1.8 Geliştirme Ortamı

ST firmasının STM32 serisi ARM Cortex çekirdeklerini kullanıyor. ST firmasının STM32F103C8T6 model mikrodenetleyicisinin olduğu küçük prototip kartını makul bir fiyata alabilirsiniz. Bu kartı programlamak için ayrıca SWD destekli bir programlayıcı modülü de almanız gerekiyor. ST-LINK/V2 klonları işinizi görecektir.(Bu yazının yazıldığı 10.06.2019 tarihinde iki modülün toplam fiyatı 50 TL civarındadır)

Eğer elinizde herhangi bir ST Nucleo, Discovery yada Eval board var ise onların üzerindeki ST-LINK'leri kullanabilirsiniz.

STM32 serisi için Proteus kısıtlı bir simulasyon desteği sağlıyor. STM32F103C8 modeli yok ama STM32F103C6 modeli mevcut. ROM/RAM yarısı ve bazı donanımların sayısı az ama test örnekleri için yeterli olacaktır.

STM chipleri için ST firmasının ücretsiz olarak sağladığı STM32CubeIDE'yi indirebilirsiniz. STM32CubeIDE Eclipse tabanlı bir IDE'dir. Eclipse CDT üzerine ST firmasının yazdığı plug-in'ler; donanım üzerinde hata ayıklama, STM için optimize newlib kütüphanesi, CubeMX entegrasyonu, vb. bir sürü ekleme mevcuttur.

Burada bulunan örnek projeler STM32CubeIDE ile oluşturulmuştur.

STM32 örnekleri için ihtiyacımız olanlar :

- STM32F103C8 prototip bordu
- SWD desteği olan bir programlayıcı (ST_LINK/V2 klonları uygundur)
- STM32CubeIDE

1.9 Blink (Göz kırpma)

Bu örnek hem çok basit hem de en zor işlemlerden biri olan blink örneğidir. Normal bilgisayar programlarında ekrana **"Merhaba Dünya"** yazdırmak ile aynı özelliktedir. Eğer bir kodu derleyip çalıştırdığınızda ekranda **"Merhaba Dünya"** yazdırabiliyorsanız çalışma ortamınız kod yazmaya hazırdır demektir. Mikrodenetleyici dünyasında **"Merhaba Dünya"**'nın karşılığı olarak ben blink kodunu denerim. Mikrodenetleyiciye blink kodunu derleyip yükleyip çalıştırdığınızda ledini yanıp sönüyorsa kod yazmaya hazırsınız demektir. Eğer yanıp sönme istediğiniz gibi değilse bir yerlerde yanlışlık vardır ve asıl kodunuzu yazmaya başlamadan önce bu sorunu bulup çözmeniz gerekir.

1.9.1 Bağlantı Şeması

Aşağıdaki şemada ATmega328p mikrodenetleyicisine bağlanmış üç adet 7 segment LED display ve iki buton bulunmaktadır. Şemada kullanılan bütün dirençler **Proteus**'da digital olarak seçilmiştir. Bu özellik ile digital bağlantılarda kullanılan dirençler simulasyon esnasında analog hesaplamalara tabi tutulmaz. Bu sayede simulasyon daha hızlı çalışır.

Dizin

Simgeler

#define, 9
#include, 8
Ön işlemci, 5

B

Başlık Dosyası, 11
Bağlayıcı, 6
Bare-metal, 4
Birleştirici Dili Dosyası, 11

C

C, 4
Compiler, 6
const, 10

D

Derleyici, 6

G

GNU Arm, 19

H

HEAP, 7

K

Kaynak Dosyası, 11

L

Linker, 6

M

MSYS2, 15

P

Preprocessor, 5

R

RTOS, 4
run-time, 7

S

Stack, 7
static, 10

V

volatile, 9